
Jira & GitHub

A Beginner's Guide for University Projects

Department of Electrical and Information Engineering
University of Ruhuna

EE6305 — Artificial Intelligence Mini Project

What this guide covers:

- Jira account setup
- Assigning work to teammates
- Logging work time
- Sprint planning
- Creating Epics, Stories & Tasks
- Connecting GitHub to Jira
- Generating reports
- Final-report evidence

No prior experience needed. This guide assumes you have never used Jira or GitHub Projects before. Every step is explained from scratch with screenshots described in words.

Contents

1	Why Bother with Jira?	2
2	Setting Up Jira (Free Plan)	2
3	Understanding Jira’s Hierarchy	2
3.1	Epics to create for EE6305	3
4	Creating Epics	3
5	Creating Tasks	3
5.1	Recommended tasks for your project	4
6	Sprints: Time-Boxing Your Work	4
7	Updating Task Status	5
8	Logging Work Time	5
9	Connecting GitHub to Jira	6
9.1	Setting up the integration	6
9.2	Linking commits to tasks	6
9.3	Recommended commit message convention	6
10	Adding Comments to Tasks	7
11	Generating Reports	7
11.1	Burndown Chart	7
11.2	Sprint Report	7
11.3	Velocity Chart	7
11.4	Time Tracking Report	7
11.5	Assignee breakdown (from GitHub)	8
12	Using Jira for Your Final Report	8
13	Day-1 Checklist	8
14	Quick Reference	9
15	Troubleshooting	9
16	Summary	9

1 Why Bother with Jira?

When you are working on a multi-week project with several teammates, it is easy to lose track of who is doing what, which tasks are blocked, and how much time you have actually spent. Jira is a project-management tool used by software teams worldwide that solves exactly these problems.

For a university project, the payoff is twofold. First, it keeps the team organised during development. Second, it generates evidence automatically — charts, time logs, commit histories — that you can paste directly into your final report to show a professional development process.

What you get for free at the end:

- A complete timeline of who did what and when
- Burndown charts showing planned vs. actual progress
- Per-member task and time reports
- Every GitHub commit linked to a task automatically

2 Setting Up Jira (Free Plan)

Jira is free for teams of up to ten people.

Step 1. Go to <https://www.atlassian.com/software/jira> and click **Get it free**.

Step 2. Sign up with your university email address. Choose **Scrum** when asked which template to use.

Step 3. Name your project. A short prefix will become your *issue key* prefix. For this project use **AI** so tasks look like **AI-1**, **AI-2**, etc.

Step 4. Invite teammates: go to **Project Settings** → **People** and add each member's email. Assign the role **Member**.

Scrum vs. Kanban: Choose **Scrum**. It supports Sprints (fixed time boxes), which map well to academic deadlines and generate better reports.

3 Understanding Jira's Hierarchy

Jira organises work in three levels. Understanding this before you create anything will save you a lot of confusion later.

Level	Jira name	What it means for your project
1 (biggest)	Epic	A major component, e.g. Vehicle Detection
2	Story	A user-facing feature inside an Epic
3 (smallest)	Task / Sub-task	A concrete piece of work, e.g. download dataset

For a technical AI project like yours, you will mostly use **Epics** and **Tasks** directly. Stories are optional but useful when a logical grouping sits between the two.

3.1 Epics to create for EE6305

Create one Epic for each major component:

1. **Setup & Data** — environment, dataset download, annotation conversion
2. **Vehicle Detection** — YOLOv8 training and baseline evaluation
3. **SAHI Integration** — sliced inference and ablation
4. **ByteTrack Tracking** — tracker integration and MOTA/IDF1 evaluation
5. **Heatmap Generation** — Gaussian accumulator and video overlay
6. **End-to-End Pipeline** — `inference.py`, Colab notebook, GitHub README
7. **Documentation** — final report, figures, results tables

4 Creating Epics

Step 1. In the left sidebar, click **Backlog**.

Step 2. Click **Create Epic** (or use the + **Create** button at the top and select Epic from the issue type dropdown).

Step 3. Fill in:

- **Summary:** e.g. *Vehicle Detection*
- **Start date** and **Due date** (use your project timeline)
- **Assignee:** leave blank for now — Epics are not assigned to one person; tasks inside them are.
- **Description:** a one-sentence description, e.g. *Fine-tune YOLOv8m on Vis-Drone2019 and evaluate baseline mAP.*

Step 4. Click **Create**. Jira gives it a key like AI-1.

Step 5. Repeat for all seven Epics listed in Section 3.

Common mistake: Do not put implementation detail in the Epic. Epics are containers. The detail lives in Tasks.

5 Creating Tasks

Step 1. Click + **Create** at the top of the page.

Step 2. Set **Issue Type** to **Task**.

Step 3. Fill in the fields:

Field	What to put
Summary	Short action phrase, e.g. <i>Convert VisDrone annotations to YOLO format</i>
Epic Link	Select the parent Epic (e.g. Setup & Data)
Assignee	The team member responsible
Story Points	Rough effort estimate: 1 (< 2h), 2 (half day), 3 (full day), 5 (2+ days)
Start date	When work should begin
Due date	When it must be done
Description	Acceptance criteria: what does “done” look like?

Step 4. Click **Create**.

5.1 Recommended tasks for your project

Below is a starting list. You should add more as the project evolves.

Epic	Key	Task summary
Setup & Data	AI-10	Set up Python venv and install all dependencies
Setup & Data	AI-11	Download VisDrone2019-DET and verify directory structure
Setup & Data	AI-12	Write VisDrone → YOLO annotation conversion script
Setup & Data	AI-13	Visually verify 20 converted labels with matplotlib overlay
Vehicle Detection	AI-14	Run 5-epoch smoke test to confirm training pipeline
Vehicle Detection	AI-15	Full 50-epoch YOLOv8m fine-tune on VisDrone train split
Vehicle Detection	AI-16	Evaluate best.pt on val split — record mAP50 and mAP50:95
SAHI	AI-17	Wrap model with AutoDetectionModel and run SAHI on val set
SAHI	AI-18	Measure SAHI mAP50 and compute % gain vs baseline
SAHI	AI-19	Sweep slice sizes 512 / 640 / 768 and record FPS vs mAP
ByteTrack	AI-20	Integrate Supervision ByteTrack into per-frame loop
ByteTrack	AI-21	Apply camera stabilisation pre-processing to test clip
ByteTrack	AI-22	Evaluate MOTA and IDF1 with motmetrics on held-out sequence
Heatmap	AI-23	Implement Gaussian accumulator buffer
Heatmap	AI-24	Add exponential decay for sliding window
Heatmap	AI-25	Alpha-blend COLORMAP_JET heatmap with original frame
Pipeline	AI-26	Write end-to-end <code>inference.py</code> script
Pipeline	AI-27	Create Colab notebook and test on T4 GPU
Pipeline	AI-28	Push to GitHub with README and usage instructions
Documentation	AI-29	Write results section with tables and qualitative figures
Documentation	AI-30	Write discussion: what worked, what failed, limitations

6 Sprints: Time-Boxing Your Work

A *Sprint* is a fixed period (usually one or two weeks) in which the team commits to completing a specific set of tasks. Sprints are the unit that generates Jira’s most useful reports.

Step 1. In the **Backlog** view, click **Create Sprint**. A new section labelled “Sprint 1” appears above the backlog.

Step 2. Drag tasks from the Backlog into Sprint 1. Start with tasks whose **due date** falls within the first two weeks.

Step 3. Click **Start Sprint**. Set the start and end dates.

Step 4. Repeat: at the end of the sprint, run a **Sprint Review** (close it in Jira) and move incomplete tasks to the next sprint.

Recommended sprint plan for a 6-week project:

Sprint	Duration	Focus
Sprint 1	Weeks 1–2	Setup, annotation conversion, smoke test
Sprint 2	Weeks 2–3	Full training, baseline evaluation, SAHI
Sprint 3	Weeks 4–5	ByteTrack, heatmap, end-to-end pipeline
Sprint 4	Week 6	Report writing, demo video, GitHub cleanup

7 Updating Task Status

Jira's board has four columns by default. Move tasks across the board as work progresses.

Column	Meaning	When to use
To Do	Not started yet	Default state when task is created
In Progress	Actively being worked on	When you start working on it
In Review	Done, waiting for teammate check	After implementation
Done	Verified complete	After review passes

Drag the task card on the board, or open the task and change the status from the drop-down at the top left. This is how Jira records your project history.

8 Logging Work Time

Logging time is how Jira generates the *Time Tracking Report* at the end, which shows how many hours each member contributed.

Step 1. Open a task (e.g. AI-23).

Step 2. Scroll down to the **Time Tracking** section.

Step 3. Click **Log work**.

Step 4. Fill in:

- **Time spent:** e.g. 3h 20m
- **Date started:** the day you did the work
- **Work description:** a one-line note, e.g. *Implemented Gaussian accumulator, tested sigma=15*

Step 5. Click **Save**.

Log as you go. Do not save all logging for the end of the project. Jira's timeline and reports depend on the dates being accurate.

9 Connecting GitHub to Jira

This is the most powerful integration. Once connected, every commit you push to GitHub will appear inside the Jira task it belongs to.

9.1 Setting up the integration

Step 1. In Jira, go to **Apps** → **Explore more apps** (top navigation).

Step 2. Search for **GitHub for Jira** and install it (it is free).

Step 3. Authorise the app to access your GitHub organisation or personal account.

Step 4. Select the repository (e.g. `aerial-traffic-detection`).

9.2 Linking commits to tasks

Once the integration is active, include the Jira issue key anywhere in your commit message. Jira will detect it automatically.

Listing 1: Commit message format

```
AI-23 Add Gaussian heatmap accumulation buffer
```

Listing 2: Example git workflow

```
git add src/heatmap.py
git commit -m "AI-23 Add Gaussian heatmap accumulation buffer"
git push origin main
```

After pushing, open task AI-23 in Jira. You will see a **Development** panel on the right side showing:

Item	What it records
Commits	Every commit referencing this task
Branches	Which branch the work was done on
Pull Requests	PRs that include this task's commits

9.3 Recommended commit message convention

Listing 3: Commit message examples

```
AI-15 Train YOLOv8m 50 epochs - mAP50 38.2 on val
AI-17 Wrap model with SAHI AutoDetectionModel
AI-21 Add affine stabiliser for drone footage
AI-26 Wire detection+tracking+heatmap into inference.py
```

Pull Request titles should also contain the Jira key: AI-20 Integrate ByteTrack into per-frame loop
This links the entire PR — including the review comments and approvals — to the Jira task.

10 Adding Comments to Tasks

Every time something important happens — a decision, a blocker, a result — add a comment to the relevant task. This is how Jira becomes a project diary.

Examples of useful comments:

- *Baseline mAP50 after 50 epochs: 38.2%. Meets target of $\geq 35\%$. Moving to SAHI next.*
- *SAHI mAP50: 51.7%. Relative gain: 35.2%. Target was 30%.*
- *Blocked: VisDrone MOT ground-truth annotations only cover 4 sequences. Using sequence 0001 for tracking eval.*

11 Generating Reports

Once your sprints have run and tasks are logged, Jira generates several reports automatically. Find them under **Reports** in the left sidebar.

11.1 Burndown Chart

Shows the total story points remaining across the sprint vs. the ideal linear burn-down. Use this to check whether the team is on track mid-sprint.

To read it: the red line is your actual remaining work; the grey line is the ideal. If the red line is above the grey, you are behind schedule.

11.2 Sprint Report

A list of every task that was **Completed** vs. **Not Completed** in a sprint, plus any tasks added mid-sprint (scope changes). Include a screenshot of this in your final report appendix.

11.3 Velocity Chart

Shows story points completed per sprint. Use this to demonstrate that the team maintained consistent productivity over the project duration.

11.4 Time Tracking Report

Go to **Reports** → **Time Tracking**. This shows:

- Estimated vs. actual hours per task
- Total hours per team member

11.5 Assignee breakdown (from GitHub)

GitHub's built-in Insights tab (**repository** → **Insights** → **Contributors**) shows commits per author over time. This complements Jira's time log with actual code-contribution data.

12 Using Jira for Your Final Report

The evidence Jira generates is directly usable in your final report. Below is a table template you can fill in from Jira's reports.

Metric	Value
Total tasks created	—
Tasks completed	—
Tasks carried over / incomplete	—
Team members	4
Total sprints	4
Total GitHub commits	—
Total pull requests	—
Total hours logged	—
Development duration	6 weeks
Task completion rate	-%

You can also include:

- A screenshot of the Burndown Chart for each sprint
- The Velocity Chart across all sprints
- A per-member time breakdown from the Time Tracking report
- GitHub's contributor graph showing commit activity over time

What this looks like to an examiner:

A project with 47 tracked tasks, 132 commits linked to Jira issues, 4 sprint reports, and a time-tracking table showing each member's contribution is immediately distinguishable from a project that just submits code and a PDF. It signals professional process and makes individual contributions transparent.

13 Day-1 Checklist

Do this in the first team meeting before any code is written.

- ☐ Create Jira project (Scrum, prefix AI)
- ☐ Invite all four members
- ☐ Create the 7 Epics listed in Section 3
- ☐ Create Sprint 1 and add the first 8–10 tasks
- ☐ Install GitHub for Jira app and connect the repository

- ☐ Agree on commit message format: **AI-XX Short description**
- ☐ Agree that everyone logs time *on the day they work*
- ☐ Start Sprint 1

14 Quick Reference

Action	How to do it
Create a task	Click + Create → Task
Link task to Epic	Set Epic Link field when creating or editing
Move to In Progress	Drag card on the board or change status dropdown
Log time	Open task → Time Tracking → Log work
Link a commit	Include AI-XX in the git commit message
Generate Sprint report	Reports → Sprint Report → select sprint
Generate time report	Reports → Time Tracking
See commit on task	Open task → Development panel (right side)
Add a comment	Open task → Comment box at the bottom
Close a sprint	Backlog → Active Sprint → Complete Sprint

15 Troubleshooting

Commits are not appearing in Jira tasks

Check that: (1) the GitHub for Jira app is installed and authorised, (2) the commit message contains the exact key in uppercase (**AI-23**, not **ai-23**), and (3) the repository is selected in the app settings.

A team member cannot see the project

Go to **Project Settings** → **People** and confirm the member was added with the **Member** role. They must accept the Atlassian invitation email first.

Burndown chart is flat or shows no data

Story points must be set on tasks *before* the sprint starts. Tasks added after the start show as scope changes, not in the original burndown.

Time tracking report shows 0h for everyone

Time must be logged via the **Log work** button inside each task. Simply updating a task's status does not count as logged time.

16 Summary

1. **Set up Jira** (free, Scrum, prefix **AI**) and invite the team.
2. **Create Epics** for each major component, then break them into Tasks.
3. **Plan Sprints** of two weeks each aligned to your project deadlines.
4. **Update status daily** by dragging task cards across the board.
5. **Log time on the day you work.** Do not leave it to the end.

6. **Use commit keys** (AI-XX) in every git commit and PR title.
7. **Add comments** on tasks whenever a result, decision, or blocker occurs.
8. **Export reports** at the end for your final report appendix.

The overhead of doing this properly is about five minutes per working day per person. The payoff is a professional, fully evidenced project history that makes your final report significantly stronger.